Fraud Detection & Customer Risk Analytics

Podsumowanie projektu portfolio: przebieg prac, poprawki i wnioski biznesowe na rozmowę rekrutacyjną

Projekt	Szczegóły
Autor	Piotr
Technologie	PostgreSQL 18, Python (pandas, numpy, matplotlib, psycopg), VS Code; Power BI (następny etap)
Dane	Syntetyczne, 10 000 klientów, 257 947 transakcji, 1.01–30.06.2026
Cel	Wykrywać podejrzane transakcje i oceniać ryzyko klientów przy jak najmniejszej liczbie fałszywych alarmów
Docelowe firmy	Fintechy i domy maklerskie, np. Revolut, XTB
Stan na	16.10.2026: baza, analiza SQL, scoring v1 i v2, analiza błędów, EDA i anomalie w Pythonie gotowe; Power BI w toku

Najważniejszy wynik w jednym zdaniu

Po analizie fałszywych alarmów i zmianie wag dwóch reguł liczba fałszywych alarmów spadła o **78%** *(1 009 → 227), a precyzja alarmów wzrosła z* **64,3% do 88,5%***, przy prawie tej samej liczbie wykrytych oszustw.*

Spis treści

1. Cel i problem biznesowy
2. Środowisko i decyzje techniczne
3. Model danych (star schema)
4. Przebieg projektu krok po kroku
5. Co poprawialiśmy po drodze
6. Najważniejsze wyniki analizy
7. Scoring ryzyka: v1 i v2
8. Statystyka kontra reguły (Python)
9. Rekomendacja dla banku
10. Wnioski na rozmowę rekrutacyjną
11. Pytania, które mogą paść, i odpowiedzi
12. Ograniczenia projektu
13. Kolejne kroki
14. Pliki projektu

1. Cel i problem biznesowy

Bank musi wyłapywać oszustwa kartowe, ale nie może blokować uczciwych klientów. Każdy błąd kosztuje, tylko w inną stronę:

Błąd	Co się dzieje	Koszt dla banku
Przepuszczony fraud (false negative)	Oszust wyprowadza pieniądze	Strata, zwroty dla klientów, ryzyko kar regulatora
Fałszywy alarm (false positive)	Blokada karty uczciwego klienta	Niezadowolony klient, infolinia, czas analityka, odejście do konkurencji

Cel projektu: zbudować przejrzysty system, który (1) wskazuje podejrzane transakcje, (2) ocenia ryzyko każdego klienta, (3) łapie jak najwięcej oszustw przy jak najmniejszej liczbie fałszywych alarmów i (4) potrafi wytłumaczyć każdą decyzję. Projekt realizuje plan „Revolut Fraud Detection & Customer Risk Analytics" (pierwotnie pod SQL Server), przeniesiony na PostgreSQL.

2. Środowisko i decyzje techniczne

Decyzja	Wybór	Dlaczego
Baza danych	PostgreSQL 18.6 (lokalnie, Windows, port 5432), baza **fraud_project**	Wcześniej pracowałem w SQL Server / SSMS. PostgreSQL jest darmowy, popularny w fintechach i w chmurze; nauka drugiego dialektu SQL poszerza warsztat.
Edytor	VS Code z rozszerzeniem PostgreSQL (pgAdmin jako zapas)	Jedno narzędzie do SQL i Pythona.
Język skryptów	Python w środowisku **.venv**	Generowanie danych, ładowanie do bazy, EDA, statystyka, eksport do Power BI.
Konwencja kodu	Cały kod po angielsku (aliasy, komentarze, komunikaty)	Projekt do portfolio dla międzynarodowych firm.
Dane	Syntetyczne, ziarno losowania 42	Prawdziwych danych bankowych nie da się pobrać; ziarno daje powtarzalne wyniki.
Wizualizacja	Power BI (robię samodzielnie)	Standard w analityce biznesowej.

Najważniejsze różnice T-SQL → PostgreSQL, z których korzystałem: **BOOLEAN** zamiast BIT, **LIMIT** zamiast TOP, **EXTRACT(HOUR FROM ...)** zamiast DATEPART, **INTERVAL '30 days'** zamiast DATEADD, **COUNT(*) FILTER (WHERE ...)** zamiast SUM(CASE ...), **CROSS JOIN LATERAL** zamiast CROSS APPLY, **RANGE BETWEEN INTERVAL ... PRECEDING** w funkcjach okna, **::date** do rzutowania, **ADD COLUMN IF NOT EXISTS**, **DROP TABLE IF EXISTS**, **COPY** do szybkiego ładowania CSV. Pełna ściągawka jest w pliku *tsql_vs_postgresql_sciagawka.md*.

3. Model danych (star schema)

Baza ma układ gwiazdy: w środku tabela faktów z transakcjami, dookoła tabele wymiarów. Klucze główne (PK) i obce (FK) pilnują spójności, np. nie da się wpisać transakcji nieistniejącego klienta.

Tabela	Wiersze	Co zawiera
dim_customer	10 000	Klient: kraj (25% PL), wiek, plan (standard/plus/premium/metal), data założenia konta (~8% kont z 2026)
dim_merchant	500	Sprzedawca: 10 kategorii, kraj (~17% spoza Europy), kategoria ryzyka
dim_device	14 511	Urządzenia klientów (65% ma jedno), w tym 263 urządzenia oszustów
dim_fx_rate	1 629	Kursy 9 walut × 181 dni, do przeliczenia kwot na EUR (PK z dwóch kolumn)
fact_transaction	257 947	Transakcje; etykieta is_fraud: 4 242 fraudy (1,64%)
fact_customer_risk	10 000	Historia klienta (previous_fraud, chargebacks) i wynik ryzyka v1 oraz v2
fact_transaction_risk	257 947	Tworzona przez SQL: 7 sygnałów, wynik i poziom ryzyka v1 i v2 dla każdej transakcji

W danych celowo są **4 scenariusze oszustw**: przejęcie konta (nowe urządzenie, inny kraj, seria dużych płatności), card testing (seria drobnych kwot w kilka minut), pojedyncza bardzo wysoka kwota i świeże konto użyte do oszustwa. Są też **legalne transakcje, które wyglądają podejrzanie** (podróże, duże zakupy), bo bez nich każdy alarm byłby trafiony, a to nierealne.

Etykieta **is_fraud** służy wyłącznie do **oceny** scoringu, nigdy do jego liczenia. W banku pochodziłaby z potwierdzonych zgłoszeń, często po tygodniach.

4. Przebieg projektu krok po kroku

Etap	Co zrobiłem	Pliki
1. Przygotowanie	Porównanie T-SQL i PostgreSQL, instalacja PostgreSQL, połączenie VS Code z bazą fraud_project, folder projektu (sql, python, data), środowisko .venv i biblioteki	tsql_vs_postgresql_sciagawka.md
2. Struktura bazy	6 tabel w układzie gwiazdy, typy danych, PK, FK, ograniczenia CHECK	00_create_tables.sql
3. Generowanie danych	Klienci, sprzedawcy, urządzenia, kursy walut, transakcje z 4 scenariuszami oszustw, historia klientów	python/01–05, 07
4. Ładowanie	TRUNCATE i COPY w kolejności wynikającej z kluczy obcych, kontrola liczby wierszy	python/06_load_to_postgres.py
5. Jakość danych	17 kontroli (duplikaty, braki, zakresy, spójność dat, osierocone rekordy, outliery); 16 OK, 1 do przeglądu (258 transakcji > 99,9 percentyla). Top 50 outlierów: 31 fraudów, 19 legalnych zakupów	01_data_quality.sql
6. Analiza SQL	Fraud rate ogółem, per kategoria, kraj, za granicą; trendy (miesiąc, tydzień, dzień, średnia krocząca 7 dni, godzina); zachowanie klientów; funkcje okna i sygnały ryzyka	02_sql_analysis/01–04
7. Scoring v1	7 sygnałów z funkcji okna, punkty według wag z planu, poziomy LOW–CRITICAL, wynik klienta z ostatnich 30 dni	03_risk_scoring.sql
8. Analiza błędów	Macierz pomyłek, tabela progów 30/50/60/70/80, sygnały w fałszywych vs prawdziwych alarmach, kombinacje sygnałów	04_false_positives.sql
9. Scoring v2	Zmiana wag dwóch reguł na podstawie analizy błędów, porównanie „przed i po”	05_risk_score_v2.sql
10. EDA w Pythonie	Kształt, typy, braki, statystyki kwot, normalne vs fraud, 5 wykresów	python/08_eda.py
11. Anomalie	IQR, z-score per klient, porównanie ze scoringiem, lift, korelacja, eksport CSV dla Power BI	python/09_anomaly_detection.py
12. Dokumentacja	Wyjaśnienia tabel, skryptów, zapytań linia po linii, notatki pytań i odpowiedzi, podsumowanie biznesowe	wyjasnienia/*.md
13. Power BI	Raport (Executive Risk Overview i kolejne strony): realizuję samodzielnie	powerbi_data/*.csv

Kolejność uruchamiania od zera:

```
00_create_tables.sql → Python 01, 02, 03, 04, 05, 07, 06 → 01_data_quality.sql → 02_sql_analysis/* → 03_risk_scoring.sql → 04_false_positives.sql → 05_risk_score_v2.sql → Python 08 → 09. Po każdym ponownym uruchomieniu 03 trzeba uruchomić 05, a potem 09.
```

5. Co poprawialiśmy po drodze

Projekt nie poszedł „od strzału”. Poniżej realne problemy, ich przyczyny i rozwiązania. To dobry materiał na rozmowę, bo pokazuje sposób myślenia, a nie tylko wynik.

Problem	Przyczyna	Rozwiązanie / czego się nauczyłem
Po uruchomieniu skryptu tabel „nic się nie utworzyło”	Okno zapytania było podpięte do domyślnej bazy postgres, a nie fraud_project; komunikat „null rows created” to normalna odpowiedź na CREATE TABLE	Usunięcie tabel z bazy postgres, nowe okno z węzła fraud_project, kontrola **SELECT current_database();** przed każdym skryptem

Problem	Przyczyna	Rozwiązanie / czego się nauczyłem
Polecenie python nie działało w terminalu	Windows kierował je do skrótu Microsoft Store	Launcher py / wyłączenie aliasów; potem stałe środowisko .venv w folderze projektu
Zmiana folderu projektu	Nowa lokalizacja i nazwa ze spacjami	Folder bez spacji, podfoldery sql, python, data; skrypty zapisują dane względem głównego folderu; nowe .venv
Trend miesięczny inny niż oczekiwany	Błąd w generatorze: normalne transakcje pomijały 1 stycznia (przesunięcie o 1 dzień), a w bazie były jeszcze stare dane	Poprawka w 05_generate_transactions.py, ponowne wygenerowanie i załadowanie; wynik: 257 947 transakcji, 4 242 fraudy, styczeń 42 334 / 563. Wniosek: zawsze porównywać wyniki z oczekiwanymi liczbami
Polskie aliasy i komentarze w kodzie	Pierwsze wersje skryptów były po polsku	Wszystkie skrypty SQL i Python przetłumaczone na angielski; dane po zmianie identyczne (sprawdzone sumą md5)
Brak danych do reguły „historia klienta”	Tabela fact_customer_risk była pusta: brak informacji o fraudach i chargebackach sprzed 2026	Nowy generator 07_generate_customer_history.py (285 klientów z previous_fraud, 807 z chargebackiem); 06 ładuje go, jeśli plik istnieje
Za dużo fałszywych alarmów w v1	Reguła „historia fraud/chargeback” (+30 pkt) była w 95,8% fałszywych alarmów	Scoring v2: historia 30 → 15, nowe urządzenie 15 → 25; fałszywe alarmy −78%
Mylące wyniki klienta po v2	Skrypt 05 nadpisywał risk_score klienta, więc v1 znikało	Osobne kolumny risk_score_v2 i risk_level_v2 (ADD COLUMN IF NOT EXISTS); obie wersje zostają w projekcie
VS Code zawieszał się na SELECT *	Rozszerzenie próbuje wyświetlić 258 tys. wierszy	Zawsze LIMIT przy przeglądaniu dużych tabel
Sygnał z 100% trafnością i kategorie z 0% fraudu	Uproszczenie generatora danych syntetycznych	Świadoma decyzja: zostawiam dane, opisuję to uczciwie w sekcji „Ograniczenia” w README

6. Najważniejsze wyniki analizy

Skala problemu

- Fraud to tylko **1,64% transakcji**, ale aż **14,65% wartości** (3,43 mln z 23,4 mln EUR).
- Średnia transakcja fraudowa to **808 EUR**, zwykła **79 EUR**: oszuści wydają rzadko, ale dużo naraz.
- Wniosek: liczenie samych transakcji zaniża problem, trzeba patrzeć też na kwoty.

Gdzie jest ryzyko

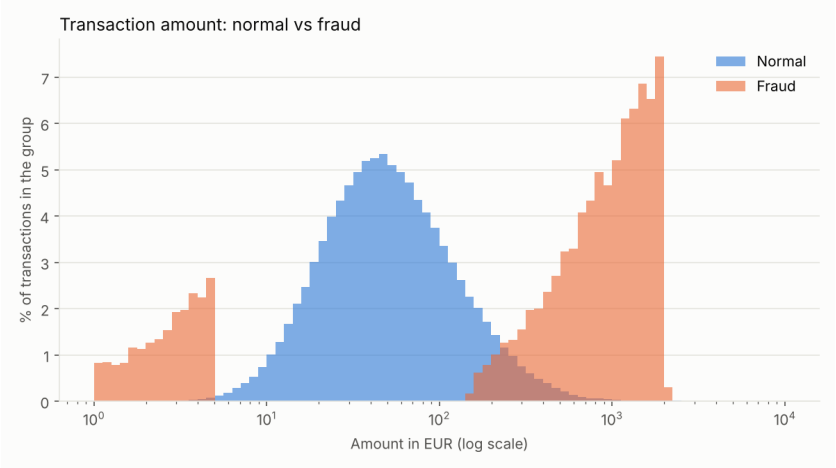
Obszar	Fraud rate	Dla porównania
Transakcje za granicą	10,14%	W kraju klienta 0,11%
Sprzedawcy spoza Europy (np. CN)	do 42,59%	Europa poniżej 1,2%
Crypto	18,97%	Zakupy online 1,23%
Elektronika / karty podarunkowe	10,21% / 8,40%	
Godzina 3:00 w nocy	najwyższy w ciągu doby	
Plan konta	1,38–1,97%	Prawie bez różnicy
Liczba urządzeń klienta (1 vs 3)	5,81% vs 18,98% klientów z fraudem	

Wniosek: ryzyko zależy od **zachowania** (gdzie, w czym, z jakiego urządzenia, jak szybko), a nie od tego, kim jest klient. Plan konta prawie nic nie mówi.

Najmocniejsze sygnały (siła sygnału = lift, czyli ile razy fraud jest częstszy, gdy sygnał się zapali)

Sygnał	Fraud rate, gdy się zapali	Lift	Ile wszystkich fraudów łapie
5+ transakcji w 10 minut	100%*	×60,8	45,2%
Nowe urządzenie (pierwsze 24 h)	43,45%	×26,4	65,8%
Ryzykowna kategoria	11,59%	×7,0	80,7%
Kwota > 3× średnia klienta	9,55%	×5,8	36,0%
Nowy kraj	8,01%	×4,9	49,2%
Historia fraud / chargeback	5,41%	×3,3	–
Nocna godzina (0–4)	3,04%	×1,9	3,9%

* efekt danych syntetycznych (opisany w ograniczeniach). Żaden sygnał sam nie wystarcza: jedne są precyzyjne, ale rzadkie, inne częste, ale nieprecyzyjne. Dlatego łączymy je w punkty.



Rozkład kwot: fraud ma dwa „garby": drobne kwoty (card testing) i duże (czyszczenie konta).

7. Scoring ryzyka: v1 i v2

Scoring to **lista 7 pytań z punktami**: za każde „tak” transakcja dostaje punkty, na końcu je sumujemy. Sygnały liczą funkcje okna w SQL (LAG, AVG/COUNT OVER, RANGE INTERVAL '10 minutes'), więc każda transakcja jest oceniana tylko na podstawie tego, co działo się **przed** nią.

Sygnał	Kolumna	v1 (plan)	v2 (poprawione)
Kwota > 3× dotychczasowa średnia klienta	s_amount_3x_avg	20	20
Nowe urządzenie (pierwsze 24 h)	s_new_device	15	**25**
Pierwsza transakcja klienta w tym kraju	s_new_country	20	20
5+ transakcji w 10 minut	s_5_tx_in_10_min	15	15
Crypto, karty podarunkowe, elektronika	s_risky_category	10	10
Wcześniejszy fraud lub chargeback	s_fraud_history	30	**15**
Godzina 0:00–4:59	s_night_hour	10	10

Poziomy: 0–29 LOW, 30–59 MEDIUM, 60–79 HIGH, 80+ CRITICAL. Wynik klienta = najwyższy wynik jego transakcji z ostatnich 30 dni (stan na 30.06.2026), bo bank chce wiedzieć, kto jest ryzykowny **teraz**.

Przykład: przejęcie konta o 2:30 w nocy, 1 500 EUR za elektronikę w Chinach z nowego telefonu daje w v2 20 + 25 + 20 + 10 + 10 = **85 pkt, CRITICAL**. Zakupy spożywcze w południe dają 0 pkt, LOW.

Jak doszedłem do v2 (analiza błędów v1, próg 60)

Próg	Alarmy	Trafione fraudy	Fałszywe alarmy	Przepuszczone	Precision	Recall
30	37 632	3 674	33 958	568	9,8%	86,6%
50	6 580	2 273	4 307	1 969	34,5%	53,6%
60	2 830	1 821	1 009	2 421	64,3%	42,9%
70	1 166	1 004	162	3 238	86,1%	23,7%
80	687	621	66	3 621	90,4%	14,6%

- **Precision** (trafność alarmów): jaki procent alarmów to prawdziwe fraudy. **Recall** (wykrywalność): jaki procent wszystkich fraudów złapaliśmy. Niższy próg łapie więcej, ale robi więcej fałszywych alarmów.
- Porównanie sygnałów: **historia** była w **95,8%** fałszywych alarmów, ale tylko w 61,1% prawdziwych, czyli głównie „robiła” fałszywe alarmy. **Nowe urządzenie** było w 10,3% fałszywych i 86,2% prawdziwych, czyli świetnie odróżniało oszustów.
- Kombinacje: zestaw kwota 3× + ryzykowna kategoria + historia dał 479 alarmów, z czego 445 fałszywych (20 + 10 + 30 = 60 pkt, dokładnie na progu). Zestaw kwota 3× + nowe urządzenie + nowy kraj + ryzykowna kategoria to 91,5% fraudu (wzorzec przejęcia konta).

Efekt zmiany wag (próg 60)

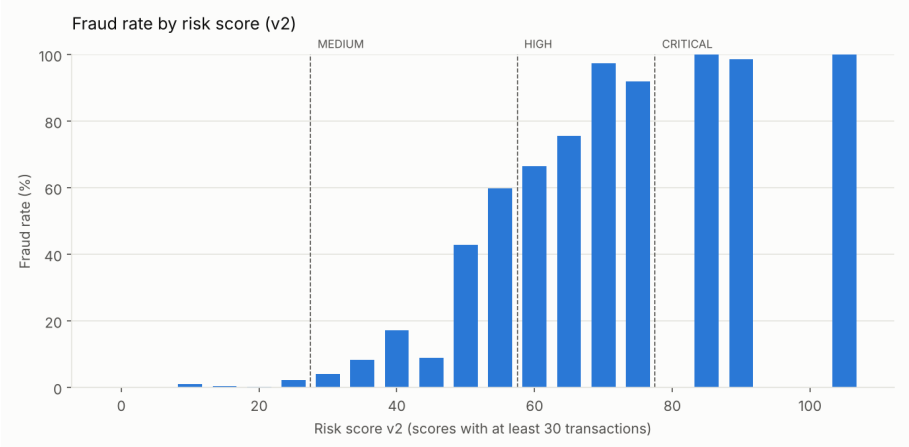
	v1	v2	Zmiana
Fałszywe alarmy	1 009	**227**	**−78%**
Wykryte fraudy	1 821	1 742	−4%
Precision	64,3%	**88,5%**	+24 pkt proc.
Recall	42,9%	41,1%	−1,8 pkt proc.
Korelacja wyniku z is_fraud	0,403	0,492	lepiej
Klienci CRITICAL / HIGH	57 / 143	24 / 68	mniej, ale trafniej

8. Statystyka kontra reguły (Python)

Sprawdziłem, czy do wykrywania fraudu wystarczy sama statystyka na kwocie, i porównałem ją ze scoringiem.

Metoda	Jak działa	Precision	Recall	Ocena
IQR na kwocie	Jeden próg dla wszystkich: Q3 + 1,5 × IQR = 191,57 EUR (22 276 alarmów)	14,8%	77,8%	Na 1 oszusta 6 zablokowanych uczciwych
z-score per klient > 3	Kwota względem średniej i odchylenia klienta	10,1%	12,8%	Słaby: fraudy same zawyżają średnią klienta
Scoring v1 ≥ 60	7 reguł, wagi z planu	64,3%	42,9%	
Scoring v2 ≥ 60	7 reguł, poprawione wagi	88,5%	41,1%	Najtrafniejsze alarmy
Scoring v2 ≥ 50	jw., niższy próg	66,1%	66,8%	Dobry kompromis do kolejki analityka

Wniosek: „nietypowa kwota” to nie to samo co „oszustwo”. Statystyka na jednej zmiennej łapie dużo, ale myli się bardzo często; reguły oparte na zachowaniu (urządzenie, kraj, tempo) są dużo trafniejsze.



Fraud rate rośnie razem z wynikiem scoringu, czyli wynik dobrze porządkuje transakcje od najbezpieczniejszych do najgroźniejszych.

Eksport do Power BI: **transactions_scored.csv** (257 947 wierszy: sygnały, wyniki v1 i v2, IQR, z-score) i **customers_risk.csv** (10 000 klientów z wynikiem v1 i v2).

9. Rekomendacja dla banku

Zamiast jednego progu proponuję **dwa poziomy reakcji** na scoringu v2:

Wynik v2	Działanie	Transakcji w 6 mies.	Ile to fraud
80+ (CRITICAL)	Automatyczna blokada i SMS do klienta	466	99,1%
50–79	Kolejka do analityka (sprawdzenie ręczne)	3 820 (ok. 21 dziennie)	ok. 62%
poniżej 50	Przepuszczamy, ewentualnie monitoring	reszta	niski

- Automatyczne blokady prawie nigdy nie trafiają w niewinnych (4 pomyłki na pół roku).
- Analityk dostaje ok. 21 spraw dziennie, z czego większość to prawdziwe oszustwa.
- Łącznie wykrywamy **2 834 fraudy (66,8%)**, ponad 1 000 więcej niż v1 przy progu 60.
- Próg to **decyzja biznesowa**: zależy od liczby analityków i od tego, co kosztuje więcej, przepuszczony fraud czy zablokowany uczciwy klient.

Czego scoring nie łapie: przy progu 50 ok. 1 400 fraudów (33%) przechodzi, bo wyglądają jak zwykłe zakupy (zwykła kwota, znane urządzenie, kraj klienta).

10. Wnioski na rozmowę rekrutacyjną

Pięć rzeczy, które warto powiedzieć

- **Liczby, nie tylko kod:** fraud to 1,6% transakcji, ale 15% wartości; najważniejsze jest zachowanie klienta, nie jego profil.
- **Najprostsza metoda to za mało:** próg kwoty dawał 85% fałszywych alarmów, więc zbudowałem scoring z 7 sygnałów zachowania.
- **Analiza błędów zamiast zgadywania:** macierz pomyłek i rozbicie fałszywych alarmów pokazały jedną przeważoną regułę (96% fałszywych alarmów).
- **Iteracja z mierzalnym efektem:** zmiana dwóch wag: −78% fałszywych alarmów, precision 64% → 88%, prawie bez straty wykrywalności. Obie wersje zostały w projekcie, więc widać „przed i po”.
- **Myślenie biznesowe:** dwa poziomy reakcji (blokada od 80, analityk od 50) i świadomość, że próg wybiera biznes.

Wersja na rozmowę (ok. 1 minuta)

„Zbudowałem w PostgreSQL i Pythonie analizę fraudu na 258 tys. transakcji. Fraud to 1,6% transakcji, ale prawie 15% wartości, i koncentruje się w transakcjach zagranicznych, crypto i elektronice. Najpierw sprawdziłem najprostsze podejście, czyli oznaczanie wysokich kwot: łapało dużo, ale 85% alarmów było fałszywych. Zbudowałem więc przejrzysty scoring z 7 sygnałów zachowania, opartych na funkcjach okna w SQL. Analiza fałszywych alarmów pokazała, że jedna reguła, historia klienta, odpowiada za 96% z nich. Po zmianie wag fałszywe alarmy spadły o 78%, a precyzja wzrosła z 64% do 88%, przy prawie tej samej liczbie wykrytych fraudów. Zaproponowałem też dwa progi reakcji: automatyczna blokada od 80 punktów i kolejka do analityka od 50.”

Umiejętności, które projekt pokazuje

Obszar	Konkretnie
SQL (PostgreSQL)	Star schema, PK/FK/CHECK, CTE, funkcje okna (ROW_NUMBER, LAG, AVG/COUNT/SUM OVER, RANGE INTERVAL), FILTER, CROSS JOIN LATERAL, VALUES, CREATE TABLE AS, ALTER/UPDATE, kontrole jakości
Python	Generowanie danych (numpy, Faker), ładowanie COPY (psycopg), EDA w pandas, IQR i z-score, wykresy matplotlib, eksport CSV
Analityka fraudu	Sygnały ryzyka, scoring regułowy, macierz pomyłek, precision/recall, lift, dobór progu, analiza fałszywych alarmów
Biznes	Koszt błędów w obie strony, rekomendacja procesu (blokada + kolejka analityka), uczciwe ograniczenia

11. Pytania, które mogą paść, i odpowiedzi

Dlaczego scoring regułowy, a nie model ML?

Reguły są przejrzyste: każdą decyzję da się wytłumaczyć klientowi, analitykowi i regulatorowi. To dobry punkt odniesienia; kolejnym krokiem byłoby porównanie z regresją logistyczną lub drzewami decyzyjnymi.

Skąd wziąłeś wagi?

v1 to wagi z planu. v2 to poprawka oparta na danych: reguła historii była w 95,8% fałszywych alarmów, a nowe urządzenie w 86,2% prawdziwych i tylko 10,3% fałszywych, więc przesunąłem punkty między nimi.

Co to precision i recall?

Precision: ile alarmów to prawdziwe oszustwa. Recall: ile wszystkich oszustw złapaliśmy. Ciągną w przeciwne strony; wybór progu to wybór kompromisu.

Jak wybrałbyś próg?

Na podstawie kosztów: ilu analityków mamy, ile kosztuje przepuszczony fraud, a ile utracony klient. Dlatego proponuję dwa progi: 80 do blokady, 50 do kolejki.

Dlaczego wynik klienta z 30 dni?

Bank chce wiedzieć, kto jest ryzykowny teraz. Stary incydent nie powinien wiecznie trzymać klienta w strefie ryzyka; 30 dni to założenie demonstracyjne do kalibracji.

Jak uniknąłeś „zaglądania w przyszłość”?

Sygnały liczą funkcje okna tylko z transakcji wcześniejszych (np. średnia klienta przed daną transakcją), a is_fraud służy wyłącznie do oceny.

Czy wyniki przeniosą się na prawdziwe dane?

Nie wprost: dane są syntetyczne, a wagi v2 oceniałem na tych samych danych. W banku sprawdziłbym je na nowym okresie (np. lipiec–grudzień) i monitorował dryf.

Co dodałbyś dalej?

Sygnały typu zmiana hasła lub telefonu przed płatnością, nowy odbiorca przelewu, „niemożliwa podróż” między krajami; model ML do porównania; dashboard monitorujący alarmy w Power BI.

12. Ograniczenia projektu (do sekcji README „Limitations”)

- Dane są **syntetyczne**; etykieta is_fraud jest pełna i pewna, a w banku byłaby opóźniona i niekompletna.
- Sygnał „5+ transakcji w 10 minut” ma 100% trafności, a 5 kategorii ma dokładnie 0% fraudu: to efekt generatora. Świadomie zostawiłem dane bez zmian i opisuję to.
- Wagi v2 dobrałem na tych samych danych, na których je oceniam (ryzyko dopasowania); w banku walidacja na nowym okresie.
- Okno 30 dni, progi poziomów (30/60/80) i testowane progi alarmów to **założenia demonstracyjne**.
- 258 transakcji powyżej 99,9 percentyla kwoty zostało w danych jako kandydaci do analizy, nie błędy.
- Scoring jest regułowy i przejrzysty, ale nie jest modelem produkcyjnym.

13. Kolejne kroki

- **Power BI** (samodzielnie): strona 1 Executive Risk Overview (fraud rate, wartość fraudu, liczba alarmów, trend), kolejne strony: analiza transakcji, ryzyko klientów, porównanie v1 i v2.
- **README** na GitHub: cel, architektura, kolejność uruchamiania, kluczowe wyniki, wykresy, ograniczenia.
- Opcjonalnie: model ML jako porównanie ze scoringiem, walidacja na nowym okresie danych.

14. Pliki projektu

Folder	Pliki
python/	01_generate_customers, 02_generate_merchants, 03_generate_devices, 04_generate_fx_rates, 05_generate_transactions, 06_load_to_postgres, 07_generate_customer_history, 08_eda, 09_anomaly_detection
sql/ (postgres/)	00_create_tables, 01_data_quality, 02_sql_analysis/01_fraud_rate, 02_fraud_trend, 03_customer_behaviour, 04_window_functions, 03_risk_scoring, 04_false_positives, 05_risk_score_v2
charts/	01–05 wykresy EDA, 06 fraud rate wg wyniku scoringu
powerbi_data/	transactions_scored.csv, customers_risk.csv
Dokumentacja (markdown)	tsql_vs_postgresql_sciagawka.md; wyjasnienia/01_tabele_i_kolumny.md, 02_skrypty_po_kolei.md, 03_notatki_pytania_i_odpowiedzi.md, 04_podsumowanie_biznesowe.md, falszywe_alarmy_v1_v2_linia_po_linii.md, outliery_linia_po_linii.md